

Data Transformation, Cataloguing & Analytics

Day 5 landed validated JSON in S3. It isn't queryable. Today you turn it into a Parquet-partitioned catalog with Glue Crawler — and immediately query it with Athena. A CATALOG CONTEXT block joins .windsurfrules (→ v3); the 6th HITL dim Schema Discipline joins the meta-checklist; an Athena workgroup with cost cap enforces query discipline.

Day 5 → Day 6

Yesterday Cascade-authored 4 Lambdas + 2 state machines that land validated JSON in S3. Today you turn that JSON into queryable Parquet, catalog it, and run reconciliation queries — all in one module.

YESTERDAY

validated/<learner>/...json files (semi-structured)

↓ becomes ↓

TODAY

validated → curated/ as partitioned Snappy-compressed Parquet, schema-versioned.

YESTERDAY

5-dim HITL meta-checklist

↓ expands to ↓

TODAY

6 dims — Schema Discipline added. Catches partition drift, schema evolution, query-cost overruns.

YESTERDAY

validator-pipeline.asl.json (3 Lambda Tasks + 2 control states + named Catch routes)

↓ extends with ↓

TODAY

Today's labs invoke ETL + Crawler MANUALLY (not yet wired into the ASL). Appending a 5th Catalog Task to the ASL is a Day 11 SAM/CI integration task.

YESTERDAY

DLQ replayer with 3 classifiers + replay_attempt

↓ unchanged ↓

TODAY

Carries forward. New failure class: SchemaInferenceError when Crawler discovers conflict.

YESTERDAY

.windsurfrules v2 with BATCH CONTEXT block

↓ promotes to ↓

TODAY

v3 — adds CATALOG CONTEXT block (table name, partition columns, Parquet vs JSON, schema version).

YESTERDAY

Lambda count: 5 of 10

↓ +1 ↓

TODAY

+1 ETL Lambda (validated → Parquet). Cumulative 6 of 10. 4 left for D8 (3 Lambdas) + D9 (1 wrapper) → cap reached on D9.

Why a catalog now — querying without one is a tax you pay forever

THE PROBLEM — JSON-IN-S3 IS A QUERY TAX

By Day 5 you have weeks of validated/ data in S3 — JSON files, partitioned by source/date. Athena CAN query JSON, but every query reads the entire file (no columnar projection), pays JSON parse cost per byte, and treats every column as a string. A simple sum-by-airline costs 100× more than the same query on Parquet.

Worse, without a catalog you've got 'schema-on-read' chaos — every consumer hardcodes field names. Add a column upstream, downstream queries silently miss it.

The catalog turns JSON-on-S3 into a real database.

THE PAYOFF — A QUERYABLE LAKE BY EOD

By 03:30 today you'll have TWO Glue databases — `accelya_validated_<learner>` (table `validated_settlement_json`, kept for cost-comparison only) and `accelya_curated_<learner>` (table `settlement`, the primary Parquet read target). JSON Crawler discovers the validated table from re-staged Hive paths; Parquet Crawler discovers the curated table from ETL output. Plus 5 reconciliation queries running through a per-learner Athena workgroup with bytes-scanned cap.

Tomorrow Day 7 designs the streaming pipeline. The curated catalog you ship today stays the primary read target for analytical queries from D7 onward.

The CATALOG CONTEXT block in `.windsurrules v3` locks Cascade into the two-table model from now on.

By the end of Day 6 you can...

CRAWL

Run a Glue Crawler over the validated JSON table path (validated/<learner>/validated_settlement_json/); observe partition discovery on second run; handle schema evolution (additive).

CONSTRAIN

Stay under sandbox Glue caps: ≤3 jobs, ≤4 DPU total, ≤6 DPU-h cumulative. Sessions disabled.

PARTITION

Choose ingest_date + source_system as the partition strategy; understand why event_date is sometimes a 3rd partition dimension.

LAYER

Apply the CATALOG CONTEXT block in .windsurfrules v3: Cascade now reads table name, partition columns, Parquet vs JSON, schema version before scaffolding any read/write to curated/.

CONVERT

Cascade-author an ETL Lambda (1 new function, count → 6) that reads validated JSON and writes partitioned Parquet (Snappy, PyArrow).

REVIEW

Apply the 6th HITL dim — Schema Discipline. Catches partition drift, additive vs breaking changes, columnar projection violations.

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Six concepts. Why catalog. Parquet 101. Crawler vs manual table. Partition strategy. Schema evolution. The CATALOG CONTEXT block in .windsurfrules v3.

TIME

~30 min

02

Guided design

Four guided slides. Catalog schema for validated/enriched/quarantine zones. Partition strategy by ingest_date+source_system. ETL Lambda contract. Sandbox-bound Glue Job design (Crawler + 1 small ETL job, ≤4 DPU).

TIME

~35 min

03

Three labs

Lab 1 — Glue Crawler over validated/, partition discovery (45 min). Lab 2 — Cascade-author ETL Lambda + Glue Job (45 min). Lab 3 — Schema evolution exercise; observe additive vs breaking (45 min).

TIME

~135 min

04

Review & wrap

Five Day-6 mistakes. Cleanup respects Glue cap. Day-6 in numbers. Bridge to Day 7 — Athena queries on this catalog.

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Six concepts. Why a catalog earns its place after weeks of JSON-in-S3. Parquet primer. Crawler vs manual table (sandbox makes this choice for you). Partition strategy. Schema evolution patterns. The CATALOG CONTEXT block in .windsurfrules v3.

Schema-on-read is fine until you have to query.

Without a catalog: every query hardcodes paths, reads whole JSON files, pays parse cost per byte, treats every column as a string. With a catalog: queries are SQL over a typed table; partition pruning and columnar projection make the same query 10–100× cheaper.

What a catalog gives you (and what it costs)

Catalog GIVES YOU:

- Typed columns (DECIMAL for fare_amount, TIMESTAMP for issue_date)
- Partition pruning (Athena reads ONLY the partitions matching WHERE)
- Columnar projection (Parquet stores per column → SELECT one col reads 1/N data)
- One source of truth for downstream consumers (no hardcoded field names)
- Schema evolution discipline (Crawler tracks added/removed columns)

Catalog COSTS YOU:

- One Glue Crawler run per ingestion cycle (~\$0.44 / DPU-hour, sandbox-capped)
- One-time ETL pass: JSON → Parquet (1 Glue Job, ≤4 DPU)
- Discipline: every consumer queries by table name, not S3 path

ECONOMICS at lab scale: query a 10MB validated/ JSON for sum-by-airline:

- Athena over JSON: full scan, ~30-100 MB DataScannedInBytes per query
- Athena over Parquet: column projected, partition pruned, ~1 MB DataScannedInBytes (Athena pricing is per TB scanned with a 10 MB minimum, so dollar amounts at lab scale are misleading. Use the bytes-scanned ratio as evidence; production cost benefit scales with data volume.)
- 100× difference at lab scale; gets worse at production scale.

01

Typed columns

DECIMAL for money. TIMESTAMP for dates. Not strings.

02

Partition pruning

WHERE ingest_date='...' reads ONLY that partition.

03

Columnar projection

SELECT one column reads 1/N of file bytes.

04

Single source of truth

Consumers query by table name, not S3 path.

05

Cost: 100× cheaper

Lab measurable. Production-grade impact at scale.

Columnar, compressed, predicate-pushdown — the canonical analytics format.

JSON is row-oriented (every row stores all columns inline). Parquet is column-oriented (every column stored separately). For analytics — which read few columns of many rows — Parquet wins by 10-100x.

Parquet anatomy (annotated reference)

```
# A Parquet file is split into ROW GROUPS (default ~128MB).
# Each row group has one COLUMN CHUNK per column.
# Each column chunk has one or more PAGES (default ~1MB).
# Each page has min/max statistics → predicate pushdown.

settlement_2026-W19.parquet
├── Row Group 0 (~128MB, ~50K rows)
│   ├── settlement_id (BYTE_ARRAY, dict-encoded) stats: min/max
│   ├── airline_code (BYTE_ARRAY, dict-encoded) stats: min/max
│   ├── fare_amount (DECIMAL(12,2)) stats: min/max
│   ├── issue_date (TIMESTAMP_MILLIS) stats: min/max
│   └── ... (one column chunk per schema column)
├── Row Group 1 (...)
└── Footer
    ├── Schema (full)
    ├── Row group locations
    └── KeyValue metadata (writer.version, etc.)

COMPRESSION CHOICES (sandbox-friendly):
  Snappy — fast, ~2-3x compression, default. CHOOSE THIS for lab.
  Gzip — slower, ~3-5x compression, CPU-bound on read.
  Zstd — best ratio, but newer; not all readers support old versions.

ROW GROUP SIZE: default 128MB is good for analytics. Smaller = more parallelism
but more metadata overhead. Lab keeps default.
```

01

Column chunks

Read one column → read one chunk per row group, not the whole file.

02

Min/max stats

Per-page stats let Athena skip pages where WHERE can't match.

03

Snappy default

2-3x compression, fast. Pick it for labs unless storage cost dominates.

04

Schema in footer

Full schema + row group locations stored at end of file.

05

Dict encoding

Repeated string columns (airline_code, source_system) compress massively.

When each wins; sandbox constraint makes Crawler the safe default.

A Glue table can be created two ways: (a) Crawler discovers schema by sampling files, (b) you write the table definition in JSON or Terraform. Sandbox: Glue ≤ 3 jobs / ≤ 4 DPU / ≤ 6 DPU-h. Crawler runs are scoped Glue 'crawler' resources, NOT Glue Jobs — they don't count against the job cap.

Crawler vs manual table — when each wins

USE A CRAWLER WHEN:

- Schema is discovered upstream and you don't fully know it (sandbox case)
- Files arrive with new partitions regularly (rerun Crawler, get new partitions)
- You want Glue to maintain schema-version history automatically
- You can tolerate a 1-3 minute discovery delay after new files land

USE A MANUAL TABLE WHEN:

- Schema is fully known and stable (production data contracts)
- You need exact partition keys (Crawler may infer them differently)
- You want IaC-managed schema (Terraform, CloudFormation)
- You need consistent column types (Crawler may infer string when DECIMAL is right)

DAY 6 CHOICE: Crawler. Reasons: sandbox forbids interactive Glue sessions; Crawler is the simplest path to a working table; Day 7 Athena will wrap quotas around it.

PRODUCTION CHOICE: usually manual + Crawler. Manual table for the published schema; Crawler for partition maintenance only (--schema-change-policy NOT_UPDATE_THE_TABLE).

01

Crawler ≠ Glue Job

Crawler is its own resource type. Doesn't count against ≤ 3 jobs cap.

02

Sessions disabled

Sandbox blocks interactive Glue sessions. Crawler-first or manual.

03

SchemaChangePolicy explicit

{UpdateBehavior=UPDATE_IN_DATABASE, DeleteBehavior=LOG}.
Update additive schema; NEVER auto-delete catalog metadata during lab recrawls.

04

Partition discovery

ingest_date=YYYY-MM-DD/source_system=BSP/ → Crawler infers both.

05

Cost

Crawler: ~\$0.44/DPU-hour. Lab run: ~5 min, ~\$0.04. Within sandbox cap.

ingest_date + source_system. Why date+source beats date alone.

Partition strategy is a contract: every query that filters on these columns gets cheap. Get it wrong and you pay full-table-scan costs forever. Right answer for airline batch: ingest_date + source_system.

Why ingest_date + source_system (and when event_date is a 3rd dim)

INGEST_DATE — the date the file landed in raw/. Most queries filter on this for daily reporting:
WHERE ingest_date = '2026-05-09' → reads ONE partition (one day)

SOURCE_SYSTEM — BSP, ARC, ATPCO, GDS. Different feeds have different schemas, cadences, costs:
WHERE source_system = 'BSP' → reads ONLY BSP partitions across all dates

TOGETHER: WHERE ingest_date = '2026-05-09' AND source_system = 'BSP' → ~1 MB DataScannedInBytes
(production cost scales linearly per TB; Athena has a 10 MB minimum per query)

WHY NOT just ingest_date?

Mixing sources in one partition forces predicate-aware queries to read all sources.
At 4 sources × 365 days × 50MB/file/day = 73GB/year, scans get expensive.

WHY NOT add EVENT_DATE as a 3rd dimension?

Tempting for late-arrival queries. But: high cardinality (one partition per event date) → small Parquet files → bad columnar performance.
When to add it: only if you have a query like 'all settlements for events in 2025-Q4 across any ingest date'. Use a date-bucketed event_year_month instead (12 partitions/year, not 365).

PARTITION KEY DEFINITION (Crawler discovers from S3 path):

s3://accelya-airline-data-ap-south-1/curated/settlement/
ingest_date=2026-05-09/source_system=BSP/part-0001.parquet

01

ingest_date

Most filters use it. Always partition by it.

02

source_system

4 distinct feeds. Partition isolates them.

03

Don't over-partition

High cardinality → small files → bad Parquet.

04

event_year_month if late

Date-bucketing (12/year) beats per-day for late events.

05

Partition keys are PATH

Crawler reads them from S3 prefix structure.

Additive vs breaking. Crawler handles additive; you handle breaking.

Schemas drift. Upstream adds a field. Renames one. Changes a type. Crawler re-runs catch some of these automatically; others need explicit handling. Know the difference.

Six schema-change patterns and how each is handled

```
# Schema before:
settlement: { settlement_id, airline_code, fare_amount }

# (1) ADDITIVE — upstream adds 'currency' column
Crawler rerun → table now has 4 columns; old rows have NULL for currency.
Athena: SELECT currency FROM ... WHERE ingest_date>'<add_date>' ✓
Schema-version bumped; readers MAY use it; writers MUST.

# (2) ADDITIVE-OPTIONAL — upstream adds 'commission_pct' (sometimes null)
Same as (1). Crawler infers DOUBLE allowing nulls.

# (3) RENAMING — upstream renames 'fare_amount' → 'amount'
BREAKING. Crawler creates a NEW column 'amount'; 'fare_amount' becomes always-null.
Fix: ETL Lambda maps inbound 'amount' back to 'fare_amount' until consumers cut over.
Or: dual-write both columns for one BSP cycle; deprecate old.

# (4) TYPE CHANGE — upstream changes 'fare_amount' from DOUBLE → STRING
BREAKING. Crawler infers most permissive type (string), Athena casts fail downstream.
Fix: ETL Lambda parses STRING → DECIMAL on read; emits DECIMAL in Parquet.
Or: reject upstream feed and notify ops via DLQ replayer.

# (5) REMOVED COLUMN — upstream stops sending 'agency_iata'
Soft-breaking. Old rows still have it; new rows write NULL.
Athena queries that filter on agency_iata still work for old data.
Decision: keep column or migrate (Day 6 lab DOES NOT migrate).

# (6) PARTITION KEY CHANGE — partition shifts from ingest_date to event_date
HARD-BREAKING. New table required (or full rewrite).
Lab: forbidden. Production: rare event, requires migration plan.
```

01

Additive = free

Crawler picks it up; old rows get NULL. No code change.

02

Rename = breaking

ETL Lambda maps old↔new for one cycle, then deprecates.

03

Type change = breaking

ETL Lambda parses + emits canonical type. Don't trust upstream.

04

Remove = soft-breaking

Old data keeps it; new is NULL. Decide: keep or migrate.

05

Partition key change

Hardest. Lab forbids. Production needs a migration plan.

Cascade now reads catalog metadata on every prompt that touches curated/ or Athena.

Day 3 gave you the 5-section prompt + .windsurfrules v1. Day 5 added a BATCH CONTEXT block (→ v2). Today you add a CATALOG CONTEXT block — promoting .windsurfrules to v3. It tells Cascade the table name, partition columns, format (Parquet vs JSON), schema version, and known evolution status.

CATALOG CONTEXT block to add to .windsurfrules (between BATCH CONTEXT and EXAMPLES)

```
## CATALOG CONTEXT
Two-table model (Lab 1 provisions both):
  Curated DB / table: accelya_curated_<learner>.settlement (Parquet, primary read target)
  JSON A/B / table: accelya_validated_<learner>.validated_settlement_json (kept for cost-
comparison only)
Format: Parquet (Snappy compression) – curated; JSON SerDe – validated
Partition keys: source_system (STRING), ingest_date (STRING – Crawler default; lab queries treat
as STRING) – Hive-style paths in S3
Schema version: v3 (current)
  v1 (2026-04): {settlement_id, airline_code, fare_amount}
  v2 (2026-05): + currency, commission_pct (additive)
  v3 (2026-W19): + ticket_number, bsp_cycle (additive)
Known evolution:
  - 'amount' inbound from some sources; mapped to 'fare_amount' by ETL
  - 'agency_iata' may be NULL for v1 rows

Read pattern (preferred):
  - SQL over the catalog table; partition pruning on ingest_date + source_system
  - SELECT only the columns needed; never SELECT *
  - Always include WHERE on at least one partition column

Write pattern (curated zone):
  - Write Parquet (Snappy), partitioned by ingest_date + source_system
  - One file per partition per ETL run; use Athena CTAS or Glue Job
  - DO NOT write JSON to curated/ (use validated/ for JSON if you need it)

Constraints (sandbox):
  - Glue Jobs ≤ 3 total, ≤ 4 DPU per job; Crawlers don't count against this cap
  - Athena workgroup quota will be set on Day 7 (cost cap per learner)
```

01

Two databases

Curated (Parquet, primary): accelya_curated_<learner>. JSON A/B: accelya_validated_<learner>. Cascade hardcodes against the curated one for production reads.

02

Partition keys explicit

Cascade always writes WHERE ingest_date AND source_system. Hive-style S3 paths.

03

Schema version tracked

Three versions so far. Cascade knows which fields are v3-required.

04

Evolution as data

Known mappings ('amount' → 'fare_amount') part of the layer.

05

Forbidden patterns named

SELECT *, no partition filter, JSON in curated/ — all banned.

Concepts complete — catalog vocabulary set

Why catalog. Parquet 101. Crawler vs manual. Partition strategy. Schema evolution patterns. The CATALOG CONTEXT block in .windsurfrules v3. Now: guided design — exact catalog schema, partition layout, ETL Lambda contract, Glue Job within sandbox cap.

VOCABULARY

Catalog database, partition pruning, columnar projection, Snappy Parquet, schema-version, additive vs breaking — every Day-6+ Cascade prompt uses these.

WHAT'S NEW

.windsurfrules v3 with CATALOG CONTEXT (CATALOG CONTEXT). 6th HITL dim (Schema Discipline). Glue Crawler as first-class artefact. Parquet replaces JSON for any analytics.

WHAT'S NEXT

Four guided design slides: catalog schema (two-table model), partition layout for the curated Parquet table (validated JSON table retained for A/B comparison), ETL Lambda contract, Glue Job design within ≤4 DPU sandbox cap.

02

PART 2 · ~35 MINUTES

Guided design

Four guided design slides. Catalog schema across validated/, enriched/, quarantine/. Partition layout — exact S3 paths Crawler will discover. ETL Lambda contract. Glue Job within sandbox cap.

Day 6 builds two databases: validated (JSON A/B comparison only) and curated (Parquet, primary read target). Day 7+ reads from `accelya_curated_<learner>.settlement`. Enriched + quarantine databases are scaffolded but not crawled today.

DATABASES (4)

Two-table model + scaffolded zones. JSON A/B for cost comparison; curated Parquet is the primary read target.

```
accelya_validated_<learner>      # JSON A/B
comparison ONLY
└─ tables/validated_settlement_json (JSON
SerDe, kept for cost evidence)

accelya_curated_<learner>        # today's
PRIMARY deliverable
└─ tables/settlement
(Parquet, partitioned, primary read target)

accelya_enriched_<learner>      # Day 9
populates (LLM enrichment)
└─ scaffold only today

accelya_quarantine_<learner>    # quarantine
reasons (Day 6 scaffold)
└─ tables/schema_drift
└─ tables/malformed
└─ tables/partial
```

TABLE: settlement (curated Parquet — primary read target)

Schema v3 — current. Partition keys at the bottom; columns above.

```
CREATE EXTERNAL TABLE
accelya_curated_<learner>.settlement (
  settlement_id  STRING,
  airline_code   STRING,
  pnr            STRING,
  fare_class     STRING,
  od_pair        STRING,
  ticket_number  STRING,      -- v3
  bsp_cycle      STRING,      -- v3
  issue_date     TIMESTAMP,
  fare_amount    DECIMAL(12,2),
  currency       STRING,      -- v2
  commission_pct DECIMAL(5,4), -- v2
  refund_status  STRING,
  agency_iata    STRING
)
PARTITIONED BY (
  -- Lab default (Glue Crawler infers): ingest_date
  STRING,
  -- Production fully-typed form (after explicit ALTER
  TABLE): ingest_date DATE.
  -- Stay consistent across queries – the labs use
  STRING,
  ingest_date  STRING,
  source_system STRING  -- BSP|ARC|ATPCO|GDS
)
STORED AS PARQUET
LOCATION 's3://accelya-airline-data-ap-south-1/curated/settlement/'
TBLPROPERTIES ('parquet.compression'='SNAPPY');
```

IAM SCOPE (TWO crawlers + ETL)

JSON crawler reads validated/<learner>, writes accelya_validated_<learner>. Parquet crawler reads curated/settlement/, writes accelya_curated_<learner>. ETL Lambda reads validated/, writes curated/. Disjoint per role.

```
# Excerpt – full policies in design doc.
# JSON crawler S3 reads:
{ "Sid": "JsonCrawlerReadValidated", "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::accelya-airline-data-ap-south-1/validated/${LEARNER}/*" }
{ "Sid": "JsonCrawlerListValidated", "Effect": "Allow",
  "Action": ["s3:ListBucket"],
  "Resource": "arn:aws:s3:::accelya-airline-data-ap-south-1",
  "Condition": {"StringLike": {"s3:prefix": ["validated/${LEARNER}/*"] }} }

# Parquet crawler S3 reads (curated only):
{ "Sid": "ParquetCrawlerReadCurated", "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::accelya-airline-data-ap-south-1/curated/settlement/*" }
{ "Sid": "ParquetCrawlerListCurated", "Effect": "Allow",
  "Action": ["s3:ListBucket"],
  "Resource": "arn:aws:s3:::accelya-airline-data-ap-south-1",
  "Condition": {"StringLike": {"s3:prefix": ["curated/settlement/*"] }} }

# Catalog writes – scoped to BOTH databases:
{ "Sid": "GlueWriteCatalog", "Effect": "Allow",
  "Action": ["glue:GetDatabase", "glue:GetTable", "glue:UpdateTable",
    "glue:BatchCreatePartition", "glue:CreateTable"],
  "Resource": ["arn:aws:glue:ap-south-1:*:catalog",
    "arn:aws:glue:ap-south-1:*:database/accelya_validated_${LEARNER}",
    "arn:aws:glue:ap-south-1:*:database/accelya_curated_${LEARNER}",
    "arn:aws:glue:ap-south-1:*:table/accelya_validated_${LEARNER}/*",
    "arn:aws:glue:ap-south-1:*:table/accelya_curated_${LEARNER}/*"] }
```

Crawler infers partition keys from S3 path structure. Get the path right, partitions are free.

CURATED LAYOUT (after ETL)

Hive-style partition keys in the path. Crawler converts these to PARTITIONED BY columns.

```
s3://accelya-airline-data-ap-south-1/curated/
└─ settlement/
  └─ table name
    └─ ingest_date=2026-05-09/
      └─ partition key 1
        └─ source_system=BSP/
          └─ partition key 2
            └─ part-00000.parquet
  └─ actual data
    └─ part-00001.parquet
      └─ source_system=ATPCO/
        └─ part-00000.parquet
    └─ ingest_date=2026-05-08/
      └─ source_system=BSP/
        └─ part-00000.parquet
```

WHAT THE CRAWLER PRODUCES

First run creates the table + initial partitions. Subsequent runs add new partitions only.

```
# After 1st Crawler run on a single day's data:
Table:
accelya_curated_<learner>.settlement
Partitions: 2 (ingest_date=2026-05-09 × {BSP, ATPCO})
Schema:      14 columns + 2 partition keys, all typed
Last crawl:  2026-05-09T11:42:00Z

# After 2nd Crawler run (next day, new data landed):
Partitions: 4 (added 2026-05-10 × {BSP, ATPCO})
Schema:      same (additive evolution if any)
Last crawl:  2026-05-10T11:42:00Z
```

QUERY PATTERN (Day 7 preview)

Athena reads ONLY the partitions that match WHERE. Without partition keys in WHERE, full scan.

```
-- GOOD: partition pruning + column projection
SELECT airline_code, SUM(fare_amount) AS total_fare
FROM accelya_curated_<learner>.settlement
WHERE ingest_date = '2026-05-09' -- STRING partition value (Glue Crawler default)
      AND source_system = 'BSP'
GROUP BY airline_code;
-- Reads ~1 MB DataScannedInBytes. Cost depends on production volume; lab evidence is the bytes ratio.

-- BAD: no partition filter
SELECT airline_code, SUM(fare_amount)
FROM accelya_curated_<learner>.settlement
GROUP BY airline_code;
-- Full table scan. Cost grows linearly with table size.
-- Day 7 workgroup will REJECT queries without partition filter.
```

One Lambda. Reads from validated/<learner>/validated_settlement_json/source_system=.../ingest_date=.../*.json (Lab 1's re-staged Hive layout), writes curated/<table>/ingest_date=...source_system=.../part-*.parquet. Triggered manually in Lab 2; Day 11 wires it into the ASL as a 5th Task.

FUNCTION CONTRACT (input/output)

Triggered manually for Day 6 lab. Day 11 wires it as a 5th Task in the validator-pipeline ASL.

```
# Input event (manually invoked or from Step
Functions Task)
{
  "file_key":
    "validated/jai/validated_settlement_json/source
_system=BSP/ingest_date=2026-05-09/m5-
001.json",
  "source_system": "BSP",
  "ingest_date": "2026-05-09",
  "chunk_id": "c0"
}

# Output
{
  "status": "converted" | "skipped" |
"failed",
  "rows_in": <int>,
  "rows_out": <int>,
  "parquet_key":
    "curated/settlement/ingest_date=2026-05-
09/source_system=BSP/part-00000.parquet",
  "file_sha256_in": "<sha256 of input JSON>",
  "file_sha256_out": "<sha256 of output
Parquet>"
}
```

CORE LOGIC (PyArrow)

PyArrow reads JSON, applies type coercion per CATALOG CONTEXT schema, writes partitioned Parquet.

```
import os, io, boto3, hashlib
import pyarrow as pa, pyarrow.parquet as pq
import pyarrow.json as paj

BUCKET = os.environ['BUCKET_NAME'] # set on Lambda; e.g. accelya-airline-data-ap-
south-1

SCHEMA = pa.schema([
  pa.field('settlement_id', pa.string(), False),
  pa.field('airline_code', pa.string(), False),
  pa.field('pnr', pa.string(), False),
  pa.field('fare_class', pa.string(), False),
  pa.field('od_pair', pa.string(), False),
  pa.field('ticket_number', pa.string(), True), # v3
  pa.field('bsp_cycle', pa.string(), True), # v3
  pa.field('issue_date', pa.timestamp('ms'), False),
  pa.field('fare_amount', pa.decimal128(12, 2), False),
  pa.field('currency', pa.string(), True), # v2
  pa.field('commission_pct', pa.decimal128(5, 4), True), # v2
  pa.field('refund_status', pa.string(), True),
  pa.field('agency_iata', pa.string(), True),
])

def handler(event, context):
    s3 = boto3.client('s3', region_name='ap-south-1')
    body = s3.get_object(Bucket=BUCKET, Key=event['file_key'])['Body'].read()
    sha_in = hashlib.sha256(body).hexdigest()

    # Parse JSON, coerce to schema, normalise inbound 'amount' -> 'fare_amount' if
    present
    table = paj.read_json(io.BytesIO(body))
    table = normalise_inbound(table) # rename + cast per CATALOG CONTEXT
    table = table.cast(SCHEMA) # raise on type drift

    out_key = make_curated_key(event) #
    curated/settlement/ingest_date=.../source_system=.../part-XXXX.parquet
    buf = pa.BufferOutputStream()
    pq.write_table(table, buf, compression='snappy', write_statistics=True)
    s3.put_object(Bucket=BUCKET, Key=out_key, Body=buf.getvalue().to_pybytes())
    return {'status': 'converted', 'rows_in': len(table), 'rows_out': len(table),
            'parquet_key': out_key, 'file_sha256_in': sha_in}

# Note: normalise_inbound() and make_curated_key() are project helpers in
src/etl_json_to_parquet/.
```

SANDBOX BUDGET

1 Lambda (count → 6 of 10). Memory 1024 MB (PyArrow needs headroom). Glue Job optional.

```
# TRAINER-PROVISIONED LAYER (do NOT build from source –
sandbox blocks native deps)
# AWSSDKPandas (PyArrow + pandas) layer ARN is trainer-
provided for ap-south-1
# Python 3.12, x86_64. Learners attach by ARN; they do not
build natively.
# Trainer publishes the exact ARN at session start; example
below is illustrative.
PANDAS_LAYER_ARN=arn:aws:lambda:ap-south-
1:336392948345:layer:AWSSDKPandas-Python312:24
# (Confirm region + runtime + architecture match before
using.)

# Lambda config
Function: accelya-etl-json-to-parquet-<learner>
Runtime: python3.12 # MUST match layer Python version
Architecture: x86_64 # MUST match layer architecture
Memory: 1024 MB # PyArrow + pandas needs headroom;
under 2048 cap
Timeout: 180s # ~50K rows/file, ~30s typical
conversion
Layers: $PANDAS_LAYER_ARN

# Why Lambda not Glue Job for the lab:
- Sandbox: Glue ≤ 4 DPU TOTAL across jobs, ≤ 6 DPU-h
cumulative
- Lambda has no DPU concept; counts only against the 10-
function cap
- 50K rows/file is well within Lambda's 15-min hard limit
- For larger files (production), use a Glue Job (1-2 DPU)
```

Optional path: a 1 Glue Job that does the same conversion, useful when files exceed Lambda's 15-min limit or you want partition-write parallelism. Stays under ≤ 4 DPU / ≤ 6 DPU-h.

GLUE JOB CONFIG (sandbox-bound)

1 job · 2 DPU · ~15 min/run · ~0.5 DPU-h/run. 12 runs/cycle fits the 6 DPU-h cap.

```
JobName:      accelya-etl-glue-<learner>
Type:         glueetl # standard Spark
Glue
GlueVersion:  4.0      # Python 3.10,
Spark 3.3
WorkerType:   G.1X     # 1 DPU per worker
NumberOfWorkers: 2     # 2 DPU total per
run; under 4 DPU cap
Timeout:      15       # minutes; safe
under 4 DPU-h
MaxRetries:   1
MaxConcurrentRuns: 1    # serial only –
sandbox-safe
ScriptLocation: s3://accelya-airline-data-
ap-south-1/scripts/etl_json_to_parquet.py

# Argument defaults
DefaultArguments:
  '--enable-job-insights': 'true'
  '--source_path':
's3://.../validated/<learner>/'
  '--target_path':      's3://.../curated/'
  '--database':
'accelya_curated_<learner>' # Parquet output
lands in curated DB
```

SCRIPT (etl_json_to_parquet.py)

Spark DataFrame: read JSON, type-coerce, write partitioned Parquet.

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv,
['JOB_NAME', 'source_path', 'target_path', 'database'])
sc = SparkContext(); glueContext = GlueContext(sc); spark =
glueContext.spark_session
job = Job(glueContext); job.init(args['JOB_NAME'], args)

# Read all validated JSON
df = spark.read.json(args['source_path'])

# Type coercion (matches the SCHEMA defined in the Lambda variant)
from pyspark.sql.functions import to_timestamp, col, when
df = (df.withColumn('issue_date', to_timestamp('issue_date'))
      .withColumn('fare_amount',
col('fare_amount').cast('decimal(12,2)'))
      .withColumn('commission_pct',
col('commission_pct').cast('decimal(5,4)'))
      .withColumn('fare_amount',
when(col('amount').isNull(),
col('amount').cast('decimal(12,2)'))
      .otherwise(col('fare_amount')))) # rename inbound
'amount'-'>'fare_amount'

# Write partitioned Parquet (Snappy)
(df.write
  .mode('append')
  .partitionBy('ingest_date', 'source_system')
  .parquet(args['target_path'] + 'settlement/',
compression='snappy'))

job.commit()
```

BUDGET MATH (per learner per day)

Lab cap math — make sure learners can't blow the cumulative 6 DPU-h bucket.

```
# Per Glue Job run (corrected per AWS DPU-hour
billing)
  2 DPU × 15 min × (1 hr / 60 min) = 0.5 DPU-
hours per run
  (Glue billed per-second with 1-minute
  minimum.)

# Per learner per day (Lab 2 + Lab 3 = up to 2
runs each)
  Lab 2: 1 run @ 0.5 DPU-h
  Lab 3: 1 run @ 0.5 DPU-h (optional – most
  learners use Lambda for additive)
  Day 6 reservation: ~1 DPU-h max

# Cumulative across the programme (sandbox cap:
6 DPU-h total)
  Headroom after Day 6: 6 – 1 = 5 DPU-h left
  for D8 + future modules
  Day 8 budget: ~1 DPU-h. 4 DPU-h still in
  reserve after that.

# Crawler (separate from Job cap)
  ~5 min × 1 DPU ≈ 0.08 DPU-h. Sandbox doesn't
  restrict Crawler-DPU explicitly
  but trainer notes ~$0.04/run is well within
  reasonable per-learner spend.
```

Catalog ratified — three artefacts ready to build against

Catalog schema (validated database, settlement table, partition keys). Curated S3 layout (ingest_date + source_system Hive-style). ETL Lambda contract + Glue Job alternative (sandbox-bound).

CATALOG SCHEMA

accelya_curated_<learner>.settlement, 14 columns + 2 partition keys, Snappy Parquet, IAM scoped per role.

CURATED LAYOUT

ingest_date=YYYY-MM-DD/source_system=BSP/part-XXXX.parquet. Crawler reads partitions from path.

ETL CHOICE

Lambda for $\leq 50K$ rows/file. Glue Job (2 DPU, 15min) for larger; both within sandbox cap.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 (45 min) — provision both databases + both crawlers; run the JSON Crawler over the validated JSON table path (validated/<learner>/validated_settlement_json/); observe partition discovery on second run.
Lab 2 (45 min) — Cascade-author the ETL Lambda + (optional) Glue Job; convert one day's JSON to Parquet.
Lab 3 (45 min) — schema evolution exercise: introduce additive change; rerun Crawler; confirm column appears with NULL for old data.

Provision TWO databases + TWO Crawlers (validated JSON + curated Parquet) + first run

BRIEF

Solo. Cascade-author CLI scripts to provision the two-table model:

(1) database accelya_validated_<learner> + JSON Crawler over validated/<learner>/validated_settlement_json/ (writes table validated_settlement_json)

(2) database accelya_curated_<learner> + Parquet Crawler over curated/settlement/ (writes table settlement)

Both crawlers stay live for the rest of D6 — they're the JSON-vs-Parquet A/B comparison surface.

Run the JSON Crawler once over the validated/<learner>/ Hive-style paths. Then upload one new day's worth of validated/ output to a new partition and rerun the JSON Crawler — observe the additive partition.

TIME

45 min

LEVEL

Working+

DELIVERABLE

scripts/provision_m6_glue.sh — creates BOTH databases + BOTH crawlers
/reviews/m6-crawler-evidence.md (first-run + second-run JSON-Crawler output side-by-side)

Six steps. Branch → .windsurfrules v3 → provision DB+Crawler → 1st run → inspect → 2nd run.

1

Branch + add CATALOG CONTEXT block to .windsurfrules

```
export LEARNER=jai
git checkout main && git pull --ff-only
git checkout -b module-6-$LEARNER
# Open .windsurfrules. Insert CATALOG CONTEXT block between
# BATCH CONTEXT and EXAMPLES. Template in Concept 6.
# Bump header comment from v2 → v3.
```

EXPECTED

On module-6-<learner>. .windsurfrules v3 — CATALOG CONTEXT block visible between BATCH CONTEXT and EXAMPLES.

2

Cascade-authors provision script (TWO databases + TWO crawlers; stable folder names ensure table names)

```
# IMPORTANT: Glue Crawler derives the table name from the LAST stable folder under its target.
# To force the table name 'validated_settlement_json', the JSON crawler targets a folder
# literally named 'validated_settlement_json/' under validated/. Same trick applies to
# 'settlement/' under curated/ for the Parquet crawler.

# In Windsurf chat (5-section prompt — your standard from Day 3):
```

EXPECTED

scripts/provision_m6_glue.sh present. Both databases and both crawlers created. Crawler targets land tables at the deterministic names validated_settlement_json and settlement.

3

Re-stage Day-5 validated outputs into Hive-style paths + run JSON Crawler

```
bash scripts/provision_m6_glue.sh $LEARNER

# Day 5's lander wrote validated/<learner>/<source>/<date>/*.json (NOT Hive-style).
# Re-stage one day's worth into the Hive layout the JSON Crawler expects, under the
# stable folder 'validated_settlement_json/' so the crawler picks the right table name:

# Targets: s3://accelya-airline-data-ap-south-1/curated/settlement/
```

EXPECTED

Day-5 outputs re-staged into Hive-style paths under validated_settlement_json/. JSON Crawler completes; accelya_validated_<learner>.validated_settlement_json present. (Curated Crawler runs in Lab 2 after ETL writes Parquet.)

4

Inspect first-run output (JSON table) + verify deterministic name

```
# Verify the table actually landed at the deterministic name 'validated_settlement_json'.
# Glue Crawler derives table names from S3 prefixes; if a duplicate name exists in the
# database, Glue may suffix with <hash>. Check + repair if needed.
aws s3 cp /trainer/data/synth/m6-day2-bsp.json \
s3://accelya-airline-data-ap-south-1/validated/$LEARNER/validated_settlement_json/source_system=BSP/ingest_date=2026-05-10/m6-day2-bsp.json \
--region ap-south-1

# If the name is anything other than 'validated_settlement_json' (e.g. a hashed suffix).
```

EXPECTED

Exactly ONE table named validated_settlement_json. Cols: settlement_id, airline_code, etc. Parts: source_system, ingest_date (Hive-style names). SerDe: a JSON SerDe.

5

Upload to a NEW Hive-style partition + rerun JSON Crawler

```
# Trainer provides synthetic file. Upload under the stable validated_settlement_json/ folder
# so the same JSON Crawler picks it up as a new partition of the same table:
aws s3 cp /trainer/data/synth/m6-day2-bsp.json \
s3://accelya-airline-data-ap-south-1/validated/$LEARNER/validated_settlement_json/source_system=BSP/ingest_date=2026-05-10/m6-day2-bsp.json \
--region ap-south-1
```

EXPECTED

JSON Crawler completes. New partition (source_system=BSP, ingest_date=2026-05-10) discovered without schema change.

6

Document evidence + commit

```
aws glue get-partitions --database-name accelya_validated_$LEARNER \
--table-name validated_settlement_json --region ap-south-1 \
--query 'Partitions[*].Values' > /tmp/parts-after-2nd-run.json
# diff this against /tmp/parts-after-1st-run.json from step 4.
# Save side-by-side to /reviews/m6-crawler-evidence.md.
git add scripts/ reviews/m6-crawler-evidence.md
```

EXPECTED

MR open on GitLab; pair partner tagged.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Two databases exist: accelya_validated_<learner> AND accelya_curated_<learner>
- Two crawlers exist: validated-json-crawler + curated-parquet-crawler (latter runs in Lab 2)
- JSON Crawler discovered table validated_settlement_json with 14 columns + 2 partition keys (source_system, ingest_date)
- validated/ uses Hive-style paths (source_system=BSP/ingest_date=YYYY-MM-DD/)
- Second JSON Crawler run added a new partition without schema change
- /reviews/m6-crawler-evidence.md captures both runs
- MR open with pair partner tagged

⚠ TOP GOTCHAS

Crawler scope too broad

Pointing at validated/ (not validated/<learner>/) crawls everyone's data. ALWAYS scope to learner.

SchemaChangePolicy missing

Lab default: UpdateBehavior=UPDATE_IN_DATABASE + DeleteBehavior=LOG. Without explicit settings, default LOG silently swallows drift.

Non-Hive paths

Crawler infers partition keys ONLY from Hive-style (key=value). validated/\$LEARNER/BSP/2026-05-10/ won't auto-infer; use source_system=BSP/ingest_date=2026-05-10/ instead.

Crawler role lacks glue:CreateTable

Crawler runs but can't create the table. Always include glue:* on the catalog.

Single crawler swap-in-place

Don't repoint the JSON crawler at curated/ later — that destroys the JSON-vs-Parquet A/B. Use TWO crawlers.

ETL Lambda + Athena workgroup + 5 reconciliation queries + JSON-vs-Parquet A/B

BRIEF

Pair-work. Cascade-authors the JSON→Parquet ETL Lambda using .windsurfrules v3 (CATALOG CONTEXT applies). Use the AWSSDKPandas Lambda layer (provided in ap-south-1). Convert one day of validated/ JSON to curated/<table>/ingest_date=...source_system=.../part-XXXX.parquet.

Then create a per-learner Athena workgroup with a query-bytes cost cap. Run 5 reconciliation queries (settlement-by-airline, missing-tickets, refund-rate, top-OD-pairs, fare-anomalies). For ONE of them, run identical SQL against the JSON table AND the Parquet table — capture DataScannedInBytes for both as your A/B cost evidence.

TIME

45 min

LEVEL

Working+

DELIVERABLE

src/etl_json_to_parquet/handler.py + tests
 Curated table
 accelya_curated_<learner>.settlement (Parquet)
 Athena workgroup accelya_<learner> with
 BytesScannedCutoffPerQuery cost cap
 /reviews/m6-query-cost-evidence.md MUST
 include:
 - DataScannedInBytes for JSON query
 (validated_settlement_json table)
 - DataScannedInBytes for Parquet query
 (curated.settlement table)
 - Reduction ratio (target: lab evidence $\geq 10\times$;
 observed often $\geq 100\times$)
 /sql/m6-reconciliation-5.sql — the 5 production-
 shaped queries

Six steps. Cascade-author Lambda → deploy → invoke → re-Crawl curated/ → workgroup + 5 queries → JSON-vs-Parquet A/B.

1

Cascade-authors ETL Lambda

```
# In Windsurf chat: continues from Lab 1 (.windsurfrules v3 with CATALOG CONTEXT applies).
# TASK: src/etl_json_to_parquet/handler.py per CATALOG CONTEXT.
# Use pyarrow + pyarrow.json. Type-coerce per SCHEMA. Map inbound 'amount'
# back to 'fare_amount'. Output one Parquet file per (ingest_date, source_system).
# Add tests/test_etl.py with a 5-row fixture.
```

EXPECTED

src/etl_json_to_parquet/handler.py + tests/test_etl.py present. Tests pass locally with pyarrow installed.

2

Package + deploy with AWSSDKPandas layer

```
(cd src/etl_json_to_parquet && zip -r ../etl.zip . && cd ../..)
# Use the TRAINER-PROVIDED layer ARN; do not build native deps.
# Trainer announces the exact ARN at session start (region+runtime+arch matched).
PANDAS_LAYER=$TRAINER_AWSSDKPANDAS_LAYER_ARN # set in your env
```

EXPECTED

Lambda Active. Cumulative Lambda count: 6 of 10 for this learner.

3

Invoke ETL on one day's data (read from Lab 1's re-staged Hive paths under validated_settlement_json/)

```
# Lab 1 re-staged Day-5 outputs under validated/$LEARNER/validated_settlement_json/.
# The ETL reads from THAT exact prefix – not from Day 5's original layout.
BASE_IN=validated/$LEARNER/validated_settlement_json
for SOURCE in BSP ATPCO; do
```

EXPECTED

Each invoke returns 200 + status:converted.
curated/settlement/ingest_date=.../source_system=.../*.parquet present.

4

Run the SEPARATE curated-parquet-crawler over curated/settlement/ (do NOT swap the JSON crawler)

```
# Lab 1 already created BOTH crawlers. Now run the curated one.
# IMPORTANT: do NOT update validated-json-crawler – that destroys the JSON-vs-Parquet A/B.
aws glue start-crawler --name curated-parquet-crawler-$LEARNER --region ap-south-1 \
while [ "$(aws glue get-crawler --name curated-parquet-crawler-$LEARNER \
```

EXPECTED

accelya_curated_<learner>.settlement present. Loc:
s3://.../curated/settlement/. SerDe:
org.apache.hadoop.hive.ql.io.parquet.serde.ParquetHiveSerDe. JSON table
accelya_validated_<learner>.validated_settlement_json untouched.

5

Create per-learner workgroup + run 5 reconciliation queries

```
# Create workgroup with cost cap (BytesScannedCutoffPerQuery = 100 MB lab default):
aws athena create-work-group --name accelya_$LEARNER --region ap-south-1 \
--configuration 'ResultConfiguration={OutputLocation=s3://accelya-airline-data-ap-south-1/athena-
results/'$LEARNER'/},BytesScannedCutoffPerQuery=104857600,EnforceWorkGroupConfiguration=true,PublishCloudWatchMetricsEnabled=true'
```

EXPECTED

Workgroup ACTIVE with cost cap. All 5 queries run SUCCEEDED; each returns within 5s.

6

JSON-vs-Parquet A/B with DataScannedInBytes evidence (poll until terminal state)

```
# Pick Q1 (settlement-by-airline). Run it twice with the SAME predicate, but each in its own DB+table:
# JSON → accelya_validated_$LEARNER.validated_settlement_json
# Parquet → accelya_curated_$LEARNER.settlement
```

EXPECTED

MR shows commits. Evidence file: JSON DataScannedInBytes ~30-100 MB;
Parquet ~1 MB → 30-100× reduction. Production cost benefit scales with
data volume (Athena charges per TB scanned).

Success criteria & common gotchas

✓ YOU ARE READY IF...

- `src/etl_json_to_parquet/handler.py` uses `pyarrow` + `AWSSDKPandas` layer
- Lambda count cumulative: 6 of 10
- `accelya_curated_<learner>.settlement` present (Parquet SerDe in `glue:GetTable`). JSON table `accelya_validated_<learner>.validated_settlement_json` untouched.
- Both Athena queries use `poll-until-terminal` (no ``sleep 3`` race); Parquet ~1 MB, JSON ~30-100 MB → $\geq 10\times$ `DataScannedInBytes` reduction
- Bytes-scanned ratio (NOT dollar amount) documented in `/reviews/m6-query-cost-evidence.md`
- MR shows 2 commits

⚠ TOP GOTCHAS

Forgetting AWSSDKPandas layer

PyArrow alone won't fit in default Lambda. Layer is mandatory.

Memory too low

PyArrow + pandas need 1024 MB for 50K-row files. 512 OOMs.

Athena no OutputLocation

Workgroup needs an `OutputLocation`. Today set via `--configuration` on `create-work-group`.

SELECT * on Parquet

VOIDS columnar projection. Always SELECT specific columns.

Partition type STRING (lab default)

Glue Crawler infers `ingest_date` as `STRING`. Lab queries treat it as a `STRING` (e.g. `WHERE ingest_date = '2026-05-09'`). Production option: `ALTER TABLE ... CHANGE COLUMN ingest_date ingest_date date` — but stay consistent across all queries.

Schema evolution (additive, JSON-side) + workgroup cap firing

BRIEF

Solo. Two demonstrations on top of Lab 1's catalog + Lab 2's workgroup.

PART A (additive schema, JSON-side): Drop a 'Day-3' file with a NEW column (`loyalty_program_id`) into `validated/` at a new Hive partition. Re-run `validated-json-crawler`. Query `accelya_validated_<learner>.validated_settlement_json`: new column appears; old partitions return NULL; new partition is populated. No ETL code change — the JSON SerDe absorbs unknown nullable fields. (Production note: Lab 2's ETL Lambda uses a fixed PyArrow schema and would drop the field; to land it in the Parquet curated table you'd need an additive-pass-through ETL update — covered as theory.)

PART B (workgroup cap fires deterministically): Temporarily DROP the workgroup's `BytesScannedCutoffPerQuery` to the Athena API minimum (10,000,000 bytes = 10 MB). Run a `SELECT *` over trainer-provided JSON test data >10 MB. Observe: `get-query-execution Status.State=FAILED` with `StateChangeReason` citing `bytes-scanned cutoff`. Use `ProcessedBytes` (with `QueryState=FAILED OR CANCELED`, `QueryType=DML`) as the CloudWatch evidence — Athena exposes `ProcessedBytes` + execution times under AWS/Athena (not a `CancelledQueryCount` metric).

TIME

45 min

LEVEL

Working

DELIVERABLE

`/reviews/m6-schema-evolution.md` (additive case on `validated_settlement_json` + ETL-update theory + fix recommendation)
`/reviews/m6-workgroup-alarm-evidence.md` (FAILED state + `StateChangeReason` + `ProcessedBytes` evidence)

Six steps. Pull Day-3 file → upload → recrawl → query NULLs → attempt rename → document fix.

1

Pull synthetic Day-3 additive file

```
aws s3 cp s3://accelya-airline-data-ap-south-1/synthetic/m6-day3-bsp-additive.json \
/tmp/m6-day3-additive.json --region ap-south-1
head -1 /tmp/m6-day3-additive.json | python3 -m json.tool | head -25
# Confirm: includes a NEW field 'loyalty_program_id' that wasn't in v3
```

EXPECTED

File downloaded. New field visible. Existing fields all present (additive).

2

Upload to a NEW Hive-style partition under validated_settlement_json/ (JSON-side; NO ETL)

```
aws s3 cp /tmp/m6-day3-additive.json \
s3://accelya-airline-data-ap-south-1/validated/$LEARNER/validated_settlement_json/source_system=BSP/ingest_date=2026-05-11/m6-day3-additive.json \
--region ap-south-1
# IMPORTANT: do NOT invoke the ETL Lambda here - its fixed PyArrow schema would drop
# loyalty_program_id. Schema evolution lands cleanly via the JSON crawler and JSON SerDe;
```

EXPECTED

File present at the new Hive partition. ETL NOT invoked.

3

Re-run JSON Crawler + verify additive evolution

```
aws glue start-crawler --name validated-json-crawler-$LEARNER --region ap-south-1
# Wait. Then:
aws glue get-table --database-name accelya_validated_$LEARNER \
--name validated_settlement_json --region ap-south-1 \
--columns 'loyalty_program_id,source_system,ingest_date'
```

EXPECTED

loyalty_program_id appears in the column list (typically STRING). Total columns: 15 (was 14).

4

Query NULL-on-old-data behaviour (against validated_settlement_json)

```
wait_done() {
  while true; do
    state=$(aws athena get-query-execution --query-execution-id $1 \
--region ap-south-1 --query 'QueryExecution.Status.State' --output text)
    echo "QueryExecutionStatus: $state"
  done
}
```

EXPECTED

Existing partitions: COUNT(*) > 0, COUNT(loyalty_program_id) = 0 (NULL). ingest_date=2026-05-11 partition: COUNT(*) > 0, COUNT(loyalty_program_id) > 0 (populated).

5

PART B — fire the workgroup cap deterministically (cap = 10 MB minimum)

```
# Drop the workgroup's BytesScannedCutoffPerQuery to the Athena API minimum (10,000,000 bytes = 10 MB).
# Lower values are rejected by the API. Lab 2 default was 100 MB; we lower temporarily to make
# the demo deterministic regardless of actual table size. Trainer-provided JSON test data
# guarantees the SELECT * scan exceeds the 10 MB cap.
aws athena update-workgroup --workgroup accelya_validated_$LEARNER --region ap-south-1 \
--bytes-scanned-cutoff-per-query 10000000
```

EXPECTED

Status.State = FAILED OR CANCELED. Status.StateChangeReason cites the bytes-scanned cutoff (the primary evidence). ProcessedBytes for that QueryState (with QueryType=DML) is non-zero in the recent window. Cap deterministically fires regardless of table size.

6

Document both parts + commit

```
# /reviews/m6-schema-evolution.md should cover:
# ## Additive change (loyalty_program_id)
# - Crawler picked it up automatically
# - Old partitions return NULL
# - No code change required
# - ETL update needed for additive pass-through (theory)
# - Lab 2's ETL uses a fixed PyArrow schema + drops loyalty_program_id
# - OR write Parquet WITHOUT casting (let Glue Crawler infer the new column)
# /reviews/m6-workgroup-cap-deterministic-evolution.md should cover:
# - Workgroup cap update required to fire the cap deterministically
```

EXPECTED

MR shows 3 commits.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- PART A — Additive column `loyalty_program_id` appears in `accelya_validated_<learner>.validated_settlement_json` after JSON Crawler rerun
- PART A — Old partitions `COUNT(loyalty_program_id) = 0 (NULL)`; new partition `COUNT > 0`
- PART A — `m6-schema-evolution.md` notes the ETL pass-through fix needed for the Parquet table
- PART B — Workgroup cap dropped to 10 MB (Athena API minimum) temporarily; `SELECT *` over trainer-provided JSON test data terminal state = `FAILED OR CANCELED`, with `StateChangeReason` citing bytes-scanned cutoff
- PART B — `AWS/Athena ProcessedBytes (WorkGroup=accelya_<learner>, QueryState=FAILED|CANCELED, QueryType=DML) > 0` in the last 10 min
- PART B — Workgroup cap restored to 100 MB at end of step
- `/reviews/m6-schema-evolution.md` + `/reviews/m6-workgroup-alarm-evidence.md` both committed
- MR shows 3 commits; pair partner tagged

⚠ TOP GOTCHAS

Forgetting partition isolation

If you upload Day-3 to an existing partition, you mix schemas. ALWAYS new partition for new data.

Crawler infers wrong type

`loyalty_program_id` may come back as `STRING` when it should be `INT`. Crawler is conservative; you can `ALTER TABLE` post-hoc.

Athena cache lag

After Crawler rerun, Athena may use cached schema for ~5 min. Force refresh: `MSCK REPAIR TABLE` or wait.

Forgetting to restore the cap

Lab 3 drops the cap to 10 MB (Athena API minimum, 10,000,000 bytes) to make Part B deterministic; restore to 100 MB at the end so Lab-2-style reconciliation queries still work.

Wrong CW metric

There is no `AWS/Athena CancelledQueryCount` metric. Use `ProcessedBytes` with the `QueryState` dimension; primary evidence is `get-query-execution Status.State + StateChangeReason`.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-6 mistakes that swallow cohorts. Cleanup respects Glue cap (don't blow remaining DPU-h budget).
Day-6 in numbers. Bridge to Day 7 — Athena queries that close the books for an airline.

Five Day-6 catalog mistakes — and what to do instead

01

Partition without matching query patterns

MISTAKE Choosing partition keys without thinking about query patterns. Partitioning ONLY by event_date creates high cardinality + tiny files. Partitioning ONLY by ingest_date hides late-event semantics.

DO INSTEAD

Default: ingest_date + source_system (pipeline/audit pruning). Keep event_date as a typed COLUMN. Add event_year_month as a 3rd partition only if real analytics query patterns justify it (low cardinality, ~12/year).

02

No compression codec

MISTAKE Default Parquet is uncompressed. Files are 3-5× larger. S3 cost up; query cost up.

DO INSTEAD

Always compression='snappy' on writes. 2-3× compression, near-zero CPU overhead.

03

Crawler over raw/

MISTAKE Pointing Crawler at raw/ exposes the whole cohort's data in YOUR catalog. IAM noise; potential cross-learner leakage.

DO INSTEAD

Crawler scope ALWAYS validated/<learner>/ or curated/<table>/. Per-learner; never cohort-shared.

04

ETL Lambda > 4 DPU equivalent

MISTAKE Lambda has no DPU concept, but if you process huge files with high memory, you exceed sandbox CPU/memory practical limits and timeout.

DO INSTEAD

≤50K rows/file in Lambda. Larger files use the Glue Job (2 DPU, 15 min).

05

Schema drift unannounced

MISTAKE Crawler updates the table silently. Downstream Day-6 Athena queries break without warning.

DO INSTEAD

Log every schema change Crawler makes. Add an SNS notification on schema-version bump (Day 10 wires this properly).

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-6-<learner> with 3 commits
- MR open; pair partner tagged
- Glue Crawler exists but DON'T DELETE it — Day 7 Athena reuses the catalog
- Curated/ Parquet preserved — Day 7 queries it
- ETL Lambda preserved — count cumulative 6 of 10
- Glue DPU-h used today: ~1 of 6 budget. 5 left for D8/future.
- AWS Bedrock Runtime invocations: 0 today (D6 ETL/SQL is Cascade-IDE work, not scripted Bedrock calls). Cumulative ~9-12 of 50 from D3+D5. Cascade IDE usage tracked separately.
- Validated/ JSON files preserved — useful for regression testing later modules

```
# Optional verification
# Lambda count (target: 6 functions named accelya-*-$LEARNER):
aws lambda list-functions --region ap-south-1 \
  --query "Functions[?contains(FunctionName,'$LEARNER')].FunctionName"

# Glue catalog state:
aws glue get-database --name accelya_validated_$LEARNER --region ap-south-1
aws glue get-table --database-name accelya_validated_$LEARNER \
  --name settlement --region ap-south-1 \
  --query
'Table.{Cols:length(StorageDescriptor.Columns),Parts:length(PartitionKeys)}'

# Glue DPU-h consumed today (sandbox cap: 6 DPU-h cumulative):
aws glue get-job-runs --job-name accelya-etl-glue-$LEARNER \
  --region ap-south-1 \
  --query
'JobRuns[*].{Started:StartedOn,DPUsec:ExecutionTime,Workers:NumberOfWorkers}'

# Bedrock budget (track all 3 metrics, per programme convention):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0

# Cross-platform UTC timestamps (works on macOS + Linux):
START=$(python3 -c 'from datetime import datetime,timezone; \

print(datetime.now(timezone.utc).replace(hour=0,minute=0,second=0,microsecond=0).is
oformat().replace("+00:00","Z"))')
END=$(python3 -c 'from datetime import datetime,timezone; \
print(datetime.now(timezone.utc).isoformat().replace("+00:00","Z"))')

for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
```

Trainer note: verify Glue DPU-h consumption in console; sandbox cap is 6 cumulative across the programme. Day 6 typically uses ~1 DPU-h. AWS Bedrock Runtime calls only count if made through the sandbox profile; Cascade IDE usage is tracked separately. Lambda count: 6 of 10. Do NOT delete the Crawler, Parquet files, or ETL Lambda — Day 7 reuses all three.

What you actually shipped

6/10

LAMBDA

+1 today (ETL JSON→Parquet). 4 left after D6 (D7 design-only uses 0; D8 +3; D9 +1 = cap reached).

1

GLUE CATALOG

accelya_curated_<learner>.settlement, 14 cols + 2 partitions, Snappy Parquet.

100×

QUERY COST CUT

Athena over Parquet vs JSON. Documented in lab evidence.

≤1 of 6

GLUE DPU-H

Up to 1 DPU-h reserved (2 runs × 0.5 each). 5 left for D8 + future.

Day 7 — Streaming Architecture on AWS

Today you closed the batch story end-to-end: validated → curated Parquet → catalog → reconciliation queries with cost evidence. Tomorrow the lens shifts to STREAMING. Kinesis Data Streams as the buffer; per-record latency budgets; the same airline data flowing one record at a time instead of one file at a time. A STREAM CONTEXT block joins .windsurfrules (→ v4): shard count, partition key strategy, retention, replay window.

FROM BATCH TO STREAM

validated/ JSON files become Kinesis Data Stream records

Same airline data, different shape. Per-record processing replaces SQS-batch. Latency budget tightens from 'minutes' to 'seconds'.

.WINDSURFRULES v4

STREAM CONTEXT block joins the v3 file

Shard count, partition key strategy, retention window, replay semantics. Cascade now reads stream metadata before scaffolding any consumer.

SANDBOX BUDGET

Day 7 uses 0 new Lambdas (design-only) + 1 Kinesis stream

D7 is design-day for streaming (mirrors how D4 was design-day for batch). Cumulative after D7: still 6 of 10 Lambdas; ~21-23 of 50 Bedrock invocations. D8 implements.

Day 7 branch: module-7-<learner>. Inherits .windsurfrules v3 (with CATALOG CONTEXT). Bring your validated catalog — D8 streaming consumer reuses the same idempotency table.

Catalog ratified. Reconciliation queries cost- bounded.

GLUE CATALOG + PARQUET

accelya_curated_<learner>.settlement. 14 cols + 2 partitions. Snappy Parquet. 1 ETL Lambda. 100× cheaper queries vs JSON.

ATHENA WORKGROUP + 5 QUERIES

Per-learner workgroup with BytesScannedCutoffPerQuery cap. 5 production-shaped reconciliation queries. Bad SELECT * cancelled by alarm — proven live.

.WINDSURFRULES v3

CATALOG CONTEXT block added. 6th HITL dim Schema Discipline. Future modules inherit. Tomorrow: STREAM CONTEXT block joins (→ v4).

Day 7 starts at 09:00. Bring your validated catalog. The streaming module begins; .windsurfrules grows a STREAM CONTEXT block (→ v4).